Data Structures and algorithms
Instructor : **Dr.Amin Eskandari**
Head-Ta's : Amir Mohammad Asadjoo, Ali Nikouan
Ta's: Mehdi Heidari, Narges Hosseini, Abolfazl Tadrisi, Arvin Afshari, Amir Arshia Tootoonchi, Fatemeh Feyzi

سوال 1:

فرض کنید در دانشگاه آزاد اسلامی واحد شیراز، وظیفه ویراستاری و آماده‌سازی یک کتاب دانشگاهی به شما محول شده است. برای انجام این کار قصد دارید یک ویرایشگر متن ساده طراحی کنید که قابلیت‌های مختلف آن با استفاده از ساختارهای داده **Stack**، **Queue** و **Linked List** مدیریت شوند. این ویرایشگر باید امکان بازگرداندن آخرین تغییرات انجام شده، مدیریت درخواست‌های ویرایشی به ترتیب ورود و نگهداری بخش‌ها یا پاراگراف‌های کتاب به گونه‌ای که بتوان آن‌ها را به راحتی اضافه یا حذف کرد، داشته باشد.
مشخص کنید برای پیاده‌سازی هر یک از این سه قابلیت، کدام یک از ساختارهای داده **Stack**، **Queue** و **Linked List** مناسب تر است و دلیل انتخاب خود را به صورت مختصر توضیح دهید.

سوال 2 :

افزودن یک گره به انتهای لیست پیوندی

در این مسئله، اشاره گری به گره ابتدایی یک لیست پیوندی و همچنین یک عدد صحیح که قرار است به لیست اضافه شود، در اختیار شما قرار می‌گیرد. یک گره جدید با مقدار داده شده ایجاد کنید، سپس این گره را در انتهای لیست پیوندی قرار دهید و در نهایت گره ابتدایی لیست پیوندی ایجاد شده را برگردانید. توجه داشته باشید که

اشاره گر head ممکن است برابر None باشد؛ این موضوع به این معناست که لیست اولیه خالی است.

تابع insertNodeAtTail را با پارامترهای زیر تکمیل کنید:

- SinglyLinkedListNode head: اشاره گری به گره ابتدایی لیست پیوندی
- int data: مقدار عددی گره‌ای که باید به لیست اضافه شود

خروجی

تابع باید مقدار زیر را برگرداند:

- SinglyLinkedListNode: اشاره‌گری به گره ابتدایی لیست پیوندی پس از اضافه شدن گره جدید

فرمت ورودی

در خط اول یک عدد صحیح n داده می‌شود که تعداد عناصر لیست پیوندی را مشخص می‌کند.

در n خط بعدی، در هر خط یک عدد صحیح قرار دارد که باید به انتهای لیست پیوندی اضافه شود.

محدودیت‌ها

$1 \leq n \leq 1000$

$1 \leq list[i] \leq 1000$

نمونه ورودی

5

141

302

164

530

474

در این ورودی، مقدار 5 نشان‌دهنده تعداد گره‌هایی است که باید در لیست قرار گیرند و مقادیر زیر به ترتیب به انتهای لیست اضافه می‌شوند:

141

302

164

530

474

نمونه خروجی

141

302

164

530

474

بنابراین پس از اجرای عملیات، لیست پیوندی نهایی به صورت زیر خواهد بود:

141 -> 302 -> 164 -> 530 -> 474 -> None

وظیفه شما این است که تابع insertNodeAtTail را به گونه‌ای تکمیل کنید که در هر بار فراخوانی، یک گره جدید با مقدار داده‌شده ساخته شده و در انتهای لیست پیوندی قرار گیرد. اگر لیست در ابتدا خالی باشد، گره جدید باید به عنوان اولین گره یا همان head لیست در نظر گرفته شود.

سوال 3 :

در سامانه خدمات دانشجویی دانشگاه آزاد اسلامی واحد شیراز، روزانه درخواست‌های مختلفی مانند صدور گواهی اشتغال به تحصیل، بررسی درخواست مهمانی، انتخاب واحد و سایر امور آموزشی توسط دانشجویان ثبت می‌شود. از آنجا که رعایت نوبت دانشجویان اهمیت دارد، درخواست‌ها باید دقیقاً به همان ترتیبی که وارد سامانه شده‌اند پردازش شوند؛ یعنی درخواستی که زودتر ثبت شده است، زودتر نیز مورد بررسی قرار گیرد. بنابراین سامانه به ساختاری با رفتار صف Queue و اصل FIFO نیاز است. با این حال، در نسخه فعلی سامانه امکان استفاده مستقیم از ساختار Queue وجود ندارد و تنها دو پشته در اختیار شما قرار گرفته است. وظیفه شما این است که با استفاده از این دو پشته، کلاسی طراحی کنید که رفتار یک صف را شبیه سازی کند. این کلاس باید شامل دو پشته باشد و متد enqueue() را برای افزودن یک درخواست جدید به انتهای صف، متد dequeue() را برای حذف و برگرداندن قدیمی ترین درخواست موجود در صف و متد front() را برای مشاهده قدیمی ترین درخواست بدون حذف آن پیاده سازی کند.

استفاده از queue.Queue، collections.deque، دستور pop(0) و دستور insert(0, ...) مجاز نیست و برای کار با دو پشته تنها باید از عملیات استاندارد append() و pop() استفاده شود. هدف این مسئله آن است که با استفاده از رفتار LIFO در دو پشته، رفتار FIFO موردنیاز یک صف ایجاد شود.

به صورت مختصر توضیح دهید که پس از پیاده سازی کلاس myqueue با استفاده از دو پشته، این روش از نظر نحوه انجام عملیات enqueue و dequeue چه تفاوتی با یک صف معمولی دارد. همچنین بیان کنید چرا در پیاده‌سازی با دو پشته ممکن است در برخی عملیات نیاز به جابه‌جایی چندین عنصر بین stack1و stack2 باشد، در حالی که در یک queue معمولی این جابه‌جایی وجود ندارد.

سوال 4 :

در یک مرکز توزیع بسته‌های پستی، بسته‌ها برای ارسال به ترتیب مشخصی در یک مسیر پردازش قرار گرفته‌اند. اطلاعات بسته‌ها به صورت یک لیست پیوندی یک‌طرفه Singly Linked List ذخیره شده است و هر گره نمایانگر یک بسته است. به دلیل تغییر در برنامه توزیع، لازم است ترتیب بسته‌ها به صورت گروه‌های kتایی معکوس شود؛ یعنی ابتدا k بسته اول، سپس k بسته بعدی و به همین ترتیب تا

انتهای لیست، در داخل هر گروه به صورت معکوس قرار گیرند.

وظیفه شما پیاده‌سازی تابع reverseKGroup(head, k) است. این تابع باید گره ابتدایی لیست head و عدد صحیح مثبت k را دریافت کرده و با تغییر ارتباط بین گره‌ها، ترتیب هر گروه kتایی را معکوس کند. در پایان نیز باید head جدید لیست را برگرداند.

برای مثال، اگر بسته‌ها به ترتیب زیر در مسیر قرار داشته باشند:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$$

و مقدار $k = 2$ باشد، ترتیب جدید باید به صورت زیر شود:

$$2 \rightarrow 1 \rightarrow 4 \rightarrow 3 \rightarrow 5$$

همچنین اگر برای همین لیست مقدار $k = 3$ باشد، نتیجه به صورت زیر خواهد بود:

$$3 \rightarrow 2 \rightarrow 1 \rightarrow 4 \rightarrow 5$$

اگر در انتهای لیست تعداد بسته‌های باقی‌مانده کمتر از k باشد، ترتیب آن‌ها نباید تغییر کند و باید به همان شکل اولیه باقی بمانند.

توجه داشته باشید که اجازه تغییر مقدار ذخیره‌شده در گره‌ها را ندارید و معکوس‌سازی باید فقط با تغییر اشاره‌گر next بین گره‌ها انجام شود.

ورودی تابع:

Head: اشاره‌گر به اولین گره لیست پیوندی

K: تعداد گره‌هایی که در هر گروه باید معکوس شوند

خروجی تابع:

اشاره‌گر به اولین گره لیست پیوندی پس از انجام عملیات معکوس‌سازی

نمونه ورودی:

5

1 2 3 4 5

2

نمونه خروجی:

2 1 4 3 5

سوال 5 :

یک پشته را با استفاده از یک آرایه با ظرفیت محدود پیاده‌سازی کنید. ظرفیت اولیه آرایه برابر ۴ است و ظرفیت هیچ‌گاه نباید از این مقدار کمتر شود.

در عملیات اضافه کردن، اگر آرایه فضای خالی داشته باشد، عنصر جدید در محل بعدی قرار می‌گیرد. اگر آرایه کاملاً پر باشد، ابتدا ظرفیت آن دو برابر می‌شود، یک آرایه جدید ساخته می‌شود، تمام عناصر آرایه قبلی به آرایه جدید منتقل می‌شوند و سپس عنصر جدید اضافه می‌شود.

در عملیات حذف، عنصر بالای پشته حذف می‌شود. پس از حذف، اگر تعداد عناصر موجود کمتر یا مساوی یک‌چهارم ظرفیت فعلی باشد، ظرفیت آرایه به نصف کاهش پیدا می‌کند. در هنگام کاهش ظرفیت نیز

باید یک آرایه جدید ساخته شده و عناصر باقی‌مانده به آن منتقل شوند.

برای مثال، اگر:

size = 8

capacity = 8

باشد و عنصر جدیدی اضافه شود، ظرفیت باید به ۱۶ افزایش پیدا کند.

همچنین اگر پس از چند عملیات حذف داشته باشیم:

size = 4

capacity = 16

چون تعداد عناصر برابر یک‌چهارم ظرفیت است، ظرفیت باید به ۸ کاهش پیدا کند.

استفاده از قابلیت تغییر اندازه خودکار آرایه یا فهرست پایتون برای انجام افزایش یا کاهش ظرفیت مجاز نیست و انتقال عناصر باید به صورت صریح انجام شود.

ساختار طراحی‌شده باید عملیات‌های زیر را پشتیبانی کند:

push x : اضافه کردن مقدار x به بالای پشته

pop : حذف و نمایش عنصر بالای پشته؛ در صورت خالی بودن پشته، None نمایش داده شود.

top : نمایش عنصر بالای پشته بدون حذف آن

size : نمایش تعداد عناصر موجود

capacity : نمایش ظرفیت فعلی آرایه

show : نمایش عناصر فعلی پشته از پایین به بالا

در خط اول ورودی تعداد دستورات q و در q خط بعدی دستورات داده می‌شوند.

1. پیچیدگی زمانی بدترین حالت یک عملیات اضافه کردن و یک عملیات حذف را مشخص کنید و توضیح دهید در چه شرایطی این بدترین حالت اتفاق می‌افتد.

2. فرض کنید هزینه اضافه کردن یا حذف یک عنصر برابر ۱ و هزینه انتقال هر عنصر هنگام تغییر ظرفیت نیز برابر ۱ باشد.

نشان دهید هزینه کل یک دنباله شامل n عملیات اضافه کردن و حذف از مرتبه O(n) است و در نتیجه هزینه سرشکن هر عملیات O(1) خواهد بود.

3. فرض کنید به جای یکچهارم، تصمیم بگیریم هر زمان تعداد عناصر به نصف ظرفیت رسید، ظرفیت آرایه نیز نصف شود.

یک دنباله از عملیات‌های اضافه کردن و حذف ارائه کنید که باعث شود ظرفیت آرایه به صورت مکرر افزایش و کاهش پیدا کند.

سپس توضیح دهید چرا این رفتار می‌تواند هزینه تعداد زیادی از عملیات‌های متوالی را افزایش دهد.

4. توضیح دهید چرا استفاده از آستانه یکچهارم برای کاهش ظرفیت، مشکل قسمت قبل را برطرف می‌کند.

به طور مشخص نشان دهید که پس از نصف شدن ظرفیت، قبل از اینکه دوباره نیاز به دو برابر کردن ظرفیت وجود داشته باشد، باید تعداد قابل توجهی عملیات اضافه کردن انجام شود.

5. فرض کنید:

n = تعداد عناصر موجود

m = ظرفیت فعلی آرایه

تابع پتانسیل زیر را در نظر بگیرید:

اگر:

$$n \geq m / 2$$

باشد:

$$\Phi = 2n - m$$

و اگر:

$$n < m / 2$$

باشد:

$$\Phi = 2m - n$$

با استفاده از این تابع نشان دهید که هزینه سرشکن عملیات اضافه کردن و حذف، حتی در صورت افزایش یا کاهش ظرفیت، از مرتبه $O(1)$ است